Part II — Planning and Measurement

The plan as a contract and the measurement as truth. Without these two things, verification is opinion — and opinion does not survive a sign-off review.

Chapters **5–7** · Continues **Part I — Foundations** · Every figure carries its year and its source

THE ARC OF PART II

Three chapters, and what each settles

CHAPTER	QUESTION	WHAT YOU LEAVE WITH
5 · Verification Planning	How does a specification become a contract?	Adversarial feature extraction, the anatomy of a vplan row, one row one metric, the review rite, risk-set depth
6 · Coverage	What does 100% actually mean?	The taxonomy, code coverage's structural blindness, model design as engineering, closure, honest exclusions
7 · Metrics-Driven Sign-off	How do we know we are done?	Five instruments, reading bug curves correctly, the sign-off checklist, blameless escape analysis

> **WHERE PART I LEFT OFF**
> Completeness is impossible, so "done" is a **risk-acceptance decision made in the open**. Part II builds the two artifacts that decision needs: the **plan** that says what was promised, and the **measurement** that says what was delivered.

SAY *The plan is written before the pressure arrives; the metrics are read while it is at its worst. Both exist to keep the same people honest.*

05

Verification Planning

Feature extraction, the vplan, traceability — and the review that turns one engineer's document into the block's contract.

Week two: no testbench, a specification, and a highlighter

> **THE SPECIFICATION SAYS**
> *"The midend legalizes each transfer against the AXI address rules, splitting any burst that would cross a 4 KB boundary."*

She interrogates it

- Split correctly for 1D transfers — and for 2D?
- For 2D the check applies per row. Does it still hold when the stride is **negative**, so rows walk *down* toward the boundary?
- Does the answer change when the **other channel** is active and the two legalizers share splitter state?

The specification is silent on all three.

> **WHAT SHE JUST WROTE**
> The plan row that, three months later, corners the worst bug in the block.
> Constrained-random will *find* it; the scoreboard will *catch* it — but the plan is what pointed both instruments at the right corner of the input space.

Nobody would have written a directed test for it, because until the plan asked the question, **nobody had imagined it.**

SAY *Planning is not a phase that consumes a finished specification. It is the activity that tests the specification, years before silicon tests the design.*

Reading the specification adversarially: three passes

A designer asks *"how do I build this?"* A verifier must ask *"how does this break?"* — systematically, with a question set per pass.

PASS	THE QUESTIONS	WHAT IT YIELDS ON THE DMA
1 · Interfaces	What transactions, what value ranges, what sequences and densities, what protocol violations must be survived, what interactions with other interfaces?	Every register per its access policy; config written mid-transfer rejected, never half-applied; legal AXI under arbitrary back-pressure
2 · Functions	Which configurations, which transformations and their sequences, the *sensitive values* that trigger each, where data must end up, how ordering is affected, how errors report?	Lengths 0, 1, one below max burst, exactly 256, one above; alignments that do and do not straddle a 4 KB frontier; strides positive, zero, negative
3 · Architecture	Can a buffer over- or underflow, where are the resource bottlenecks, can requests collide on them, can one path block another?	Two channels contending for one backend port; one channel's error not perturbing the other; **splitter state shared between channels**

> CLASSIFY WHAT YOU COLLECT — FOUR BINS
>
> **Isolated features** (one behaviour at a time) · **mixed features** (the key combinations, where interconnect and multi-channel bugs live) · **legal exceptions** (soft reset mid-transfer, error responses, FIFO-full) · **illegal scenarios** (what "unsupported" means: ignored, or recovered by reset). *The last two are chronically under-populated in first drafts.*

SAY *The killer question — negative stride, near the boundary, while channel 1 is active — belongs to pass 3. It could not come from the functional spec, because shared midend state is an implementation fact.*

What must not happen — and what is out of scope

MUST NOT HAPPEN

Verification detects only the errors it is looking for. A feature list of affirmative sentences is **half a list**. Enumerate the relevant and probable errors — there are infinitely many ways to fail, so judgment selects which the environment must be able to see.

Affirmative: "data is delivered uncorrupted, in order, exactly once."
Its negative twin: *no transfer shall ever write a byte outside its legalized destination window, under any channel interleaving.*

That negative twin is what the scoreboard's address check implements — and it, not any directed expectation, is what fired on the negative-stride bug.

OUT OF SCOPE

Conditions the design need not survive, **written down**, so that "we do not verify descriptor chains longer than memory" is a reviewed decision rather than an accident.

Interviewing the designer

- *"What was the hardest part to get right?"* → instant plan priority
- *"What state is shared between the channels?"* → decides whether channel isolation is one feature or ten
- *"What did you assume software will never do?"* → each answer is either a signed out-of-scope entry, or a new feature — never an unexamined hope

ONE BOUNDARY TO HOLD

"If it is not specified, do not verify it" applies to **genuine don't-cares**. An *incomplete* specification is not a don't-care — it is a defect to be fixed.

The DMA feature list, first cut

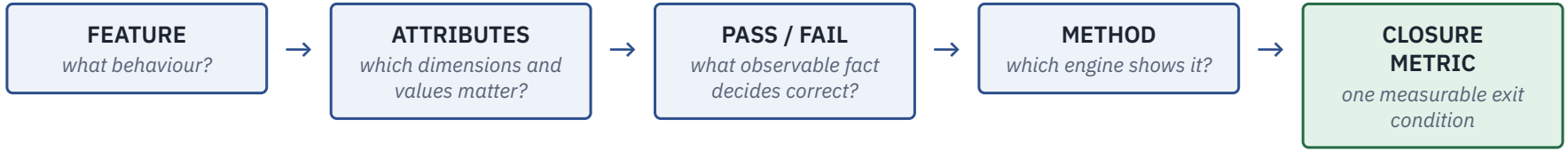
```
DMA-F01  Frontend: register access per access policy (RO/RW/W1C), reset values
DMA-F02  Frontend: config write during active transfer rejected, error flagged
DMA-F03  Midend: 1D transfer legalized into AXI bursts, max 256 beats
DMA-F04  Midend: burst split at 4 KB boundary, all alignments, len 0/1/max+-1
DMA-F05  Midend: 2D transfer, per-row legalization, stride > 0, = 0, < 0
DMA-F06  Midend: 4 KB split correct under 2D negative stride  [arch pass + interview]
DMA-F07  Midend: channel 0/1 legalizer state fully independent  [interview]
DMA-F08  Backend: AXI protocol compliance under arbitrary back-pressure
DMA-F09  Backend: SLVERR/DECERR reported in the issuing channel's status only
DMA-N01  MUST NOT: write outside legalized destination window (any interleaving)
DMA-N02  MUST NOT: raise done interrupt before last write response accepted
DMA-N03  MUST NOT: propagate channel A error into channel B stream
DMA-X01  OUT OF SCOPE: descriptor fetch from uncached device memory (not supported;
         software constraint documented, reviewed with design in week 3)
```

Note the shape: affirmative features `F`, must-not-happen features `N`, out-of-scope entries `X` with a review date — and **provenance tags** where a feature came from the architecture pass or the interview rather than the spec text.

THE AUDIT

Count the features tagged `[interview]` and `[arch pass]`. **Zero means extraction never happened** — the plan is the specification's headings with "verify that" prepended.

Anatomy of a vplan row



Conditions plus expected result, never an implementation description. Unique ID, cross-referenced to the spec clause — and retired, never reused, when the requirement dies.

Control field → enumerate every value. Data field → choose by how the design processes it. Pattern: **boundary, both neighbours, the far extreme — crossed with every configuration that moves the boundary.**

"Transfer completes" is not a criterion. This field is a **checker specification**: it says which errors to look for, so an engineer who is not the block's expert can build a reliable check.

Match to the failure signature: signal-level → assertion; per-transaction → scoreboard; statistical → offline trace analysis; local invariant → formal; and some rows close by *review*.

A covergroup at 100%, a proven property, a passed directed test, a signed review.

TRACEABILITY IS THE SAME LINK READ BACKWARDS

Each ID cross-references the spec clause it came from, the tests and properties that discharge it, and the coverage points that measure it. **A feature with no cross-reference is not being verified** — and the plan is the only place that fact is visible *before* tape-out.

SAY *Put the row's ID in the checker's error message, so a failure names the requirement it violated.*

One row, one metric — two features become four rows

ID	FEATURE & ATTRIBUTES	PASS / FAIL CRITERION	METHOD	CLOSURE METRIC
DMA-F06a	2D legalization at the 4 KB boundary. Stride sign {+, 0, −} × headroom to the next frontier {> a burst, = a burst, < a burst, straddling} × burst length {1, 255, 256} × other channel {idle, active}	Every AXI burst issued lies within one 4 KB frame; the union of the bursts equals the transfer's exact byte set	Random sim, scoreboard address and data check	Covergroup `cg_2d_4kb` at 100%
DMA-F06b	Splitter never emits a boundary-crossing burst, for any input	The AXI4 4 KB rule as a safety property on the backend's write-address channel	Formal, backend standalone	`p_no_4kb_crossing` proven unbounded
DMA-N01a	No write outside the legalized destination window, any channel interleaving	Scoreboard flags any write beat outside the expected window, with the feature ID in the message	Random sim (always-on check)	Check enabled in 100% of regression runs
DMA-N01b	That check demonstrably fires	Injection drives one write beat past the window's edge; the failure condition is the **absence** of the error message	Directed sim, error injection	Injection test passes

WHY SPLIT, RATHER THAN JOIN WITH A SEMICOLON

"Check enabled everywhere *and* proven to fire" is two kinds of evidence: a tool evaluates each separately and neither together, so a joined row closes by fatigue. The split also houses a second rule — **never trust a checker you have not seen fail.**

SAY *F06b names a binding point, not just a property: bind the same text inside the midend and the row closes while the block is still wrong.*

72 combinations. The bug lives in two of them.

```
covergroup cg_2d_4kb @(posedge clk iff midend_txn_done);
  cp_stride_sign : coverpoint txn.stride_sign
    { bins pos = {POS}; bins zero = {ZERO}; bins neg = {NEG}; }
  // txn.bnd_rel: per-row relation, computed by the monitor from
  // the row's headroom (bytes to the next 4 KB frontier, in the
  // stride's direction) and this row's burst bytes. The four
  // values partition every row once.
  cp_boundary_dist : coverpoint txn.bnd_rel
    { bins gt_burst = {GT_BURST};
      bins eq_burst = {EQ_BURST};
      bins lt_burst = {LT_BURST};
      bins straddle = {STRADDLE}; }  // must be split
  cp_len : coverpoint txn.burst_len
    { bins one = {1}; bins max_m1 = {255}; bins max = {256}; }
  cp_other_ch : coverpoint other_channel_active;
  x_worst : cross cp_stride_sign, cp_boundary_dist,
                  cp_len, cp_other_ch;
endgroup
```

3 × 4 × 3 × 2 = **72** combinations, each nameable in specification vocabulary.

stride · boundary	len 1 idle	len 1 busy	len 255 idle	len 255 busy	len 256 idle	len 256 busy
pos · gt_burst						
pos · eq_burst						
pos · lt_burst						
pos · straddle					×	×
zero · gt_burst						
zero · eq_burst						
zero · lt_burst						
zero · straddle					×	×
neg · gt_burst						
neg · eq_burst						
neg · lt_burst						
neg · straddle		■		■	×	×

The industry's accumulated DMA post-mortems live in `neg` × `straddle` × other-channel-active — **two cells**. The third is not missing coverage: the solver budget caps a row at 256 beats, so a straddling row is *split* into pieces none of which can be the 256-beat maximum. The whole `straddle` × `len 256` column (×) is excluded by **that budget, not by geometry** — a recorded choice: lift it and the column returns.

> **AND THE PAIRING RULE**
>
> This coverage means something only because the scoreboard check of `DMA-N01a` runs in the same simulations. Coverage without checking certifies stimulus, not correctness.

The plan review: three blades, applied row by row

A review that walks the feature list nodding is a signing ceremony. The published name for the procedure is **weakness analysis**.

BLADE	THE CHALLENGE	FAILING ROWS
Correctness	Is this row real? Does the DUT implement this option, is the behaviour fully specified, do the parameters allow it, is it a valid use case — do we care?	Are **deleted**. Deleting rows is a review success: unverifiable and irrelevant rows cost real schedule
Precision	What exactly is verified, in what context? Dependencies, disqualifying conditions, value ranges, event timing — and different goals in different contexts?	"Verify 2D transfers" fails. `DMA-F06a` 's attribute list passes
Completeness	For each influencing variable, how can it change and what does that do? For each scenario, what do you expect *and what do you not expect*? Which combinations need their own rows?	Produce new rows — which is what the outsiders in the room are for

> **SCENARIO — THE OBJECTION THAT MUST NOT WIN**
>
> The designer objects to `DMA-F06a` : *"that can't happen — the driver library always allocates row-aligned buffers."* Two questions settle it. **Is that library the only software that will ever program this block?** (No.) **Does the RTL check the condition, or assume it?** (Assume.) The row stays, its risk rank *rises*, and a must-not-happen row is added.

SAY *The objection and its resolution are recorded on the row. Those records outlive both engineers — they are the review testing the plan against knowledge that exists only in people.*

Risk sets depth, feature by feature

A plan that assigns every feature the same depth has not been planned; it has been transcribed. Depth is bought with **consequence × likelihood**.

Five drivers, scored per feature

DRIVER	DMA-F06
Novelty (new logic vs silicon-proven reuse)	`HIGH` new 2D negative-stride path
Structural complexity (shared state, concurrency)	`HIGH` splitter state shared across channels
Specification quality	`HIGH` the document is silent
Controllability / observability at this level	`LOW` good at block level
Cost of escape	`WORST` silent corruption, no flag

Four drivers high — and the favourable one only makes depth *affordable*, not unnecessary. Run the same five questions on `DMA-F01` and every answer inverts.

What the ranking buys

FEATURE	PLANNED DEPTH
F01 register access	Generated register tests; review of the generator config
F03 1D legalization	Random sim + scoreboard; boundary-value coverage
F06 2D neg-stride @ 4 KB	**Two rows: full cross in random sim + formal safety property**
N01 window violation	Two rows: always-on scoreboard check + its injection test

PRIORITY PRE-DRAWS THE CUT LINE

Must-have defines first-time success and is verified thoroughly. **Should-have** gets basic operation, depth as time allows. **Nice-to-have** is verified "as time allows" — which today's schedules almost guarantee means never. That is the point: the cut line is drawn in daylight, months early.

DISCUSSION

Questions for the room — the plan

1. Open your current plan. How many rows came from the **architecture pass or a designer interview** rather than the specification's headings?
2. Pick any row. Can you name its **single closure metric** in one breath? If it needs an "and", it is two rows.
3. Which of your checkers has **never been seen to fail**? What would it take to write its injection test this week?

FACILITATION *Question 3 usually produces an uncomfortable pause. An unfired checker is a testbench feature nobody has verified.*

06

Coverage: Theory and Practice

The instrument that measures what your tests never did — and the discipline of trusting it only as far as it deserves.

The bug that walked through 100% code coverage

Two weeks of clean regressions. Code coverage on the accelerator RTL: **100%** — every line, every branch. Then a firmware engineer programs a job nobody imagined: **2-bit weights**, a tensor **one pixel wide**, launched while the DMA streams camera frames through the same SRAM banks.

Starved by contention, the weight prefetcher wraps its buffer pointer one entry early. The output feature map is subtly wrong.

> THE QUESTION NOBODY COULD ANSWER
>
> **What did the tests never do?** Code coverage says "every line executed." It cannot say "every configuration exercised" — the missing scenario was not a line of RTL but a *combination*: precision × geometry × contention.

Two axes, four coverage spaces

	FROM THE IMPLEMENTATION	FROM THE SPECIFICATION
Implicit inherent in a representation	Code and structural coverage `FILLED — and green`	Metrics inferred from a natural-language spec `no practical implementation`
Explicit invented by the engineer	Functional coverage of microarchitecture — the prefetch buffer's occupancy `EMPTY — where the bug lives`	Functional coverage from the spec — precision, geometry, contention `EMPTY`

Three quadrants empty, and the one that was filled embodies nobody's judgment.

SAY *Implicit metrics come for free and embody nobody's judgment. Explicit ones cost effort and are exactly as good as the engineering behind them. Ask of any metric: whose judgment is this?*

CHAPTER 6 · THE FREE METRIC

Code coverage: free, necessary, structurally blind

SUB-METRIC	WHAT IT CATCHES	ITS BLIND SPOT
Line / statement	Dead configuration branches, unexercised error legs, whole features no test reached	Why or in what context a line ran — executed once, under the friendliest conditions, is "covered"
Branch	Each decision point taking both directions — strictly stronger than statement	**Paths.** A suite can hold 100% statement coverage while covering only 75% of control paths
Condition / expression	Whether each term independently caused the decision — `parity==ODD || parity==EVEN` reads 100% statement, 50% expression	Reaching 100% expression coverage is itself extremely difficult
Toggle	Bit-level activity: integration sanity, power-intent connectivity — the only metric that sees inside structures the others treat as one object	The coarsest of the family; says nothing about behaviour
FSM	Visited states and traversed arcs	A 5-state machine with 14 transitions hits 100% *state* coverage from one sequence covering **36%** of transitions — and the graph is extracted from the implementation, so it needs review against the spec

THE SHARPEST FORMULATION OF THE BLINDNESS

"The code in an empty module is always 100% covered." If the accelerator's designer never implemented the 2-bit unpacking mode, no code metric reports it missing. **Low numbers are a reliable alarm; a high number is no indication that the job is over.**

SAY *Its holes are meaningful; its fullness is not. Code coverage gates the floor. Functional coverage against the plan is the ceiling.*

Functional coverage inverts the economics

The vocabulary

covergroup	The container: a user-defined type encapsulating a coverage model
coverpoint	One variable or expression to observe
bins	Counters partitioning that variable's values into the cases worth counting
cross	The Cartesian combinations — where most interesting bugs live
ignore_bins	"Not my job" — removes values from the space
illegal_bins	"Must never happen" — raises a runtime error, taking precedence over any other bin

Keep that last asymmetry: the second is a **checker wearing coverage syntax**.

> **ONE LANGUAGE DEFAULT TO SUSPECT IMMEDIATELY**
>
> A coverpoint with no bins gets them automatically: one per value for small domains, otherwise the range sliced uniformly into at most `auto_bin_max` bins — **default 64**.
>
> Slice the accelerator's 9-bit tensor-width field into 64 uniform bins and each is eight values wide, so the boundary value **16** — the MAC lane count, where behaviour changes — lands in an anonymous bucket with widths 17 to 23. The tool has just merged the two the datapath treats differently.

Three families, three sources of truth

- **Code coverage** asks the implementation: *"was all of you exercised?"*
- **Functional coverage** asks the specification: *"did everything you promise actually happen?"* — data coverage in covergroups, activity coverage in `cover property`
- **Assertion coverage** asks the checkers: *"were you there when it did?"* — an assertion whose antecedent never occurred has verified nothing

What "100%" means — and what it never can

Every coverage percentage is a fraction, and the denominator is the part people forget. It is not "everything the block can do." It is **"everything the model's author thought to enumerate."**

> **CHAPTER 3 COLLECTING ITS DEBT**
>
> A functional coverage model *is* the equivalence-class partition, made executable: **bins are equivalence classes with a counter attached.** So it inherits the partition's property — a lossy, chosen projection of an intractable space.

Fidelity, and the bug's footprint

- **Fidelity** = how well the model captures the actual behavioural requirements. 18 specified register values as 18 bins: no gap. 2^{32} addresses in 16 ranges: a substantial one. Omitting *relationships* loses fidelity too.
- **Footprint:** some bugs occupy a huge region — any test in the right mode trips them. Others occupy a *point*: one instruction, one fault, one interrupt, coincident.
- A high-fidelity model **demands** the bug-exposing scenario. A low-fidelity one files it with its harmless neighbours — so fidelity is *spent* where the risk analysis says the bugs are.

> **THE MOST IMPORTANT DEFLATION IN THE CHAPTER**
>
> **Coverage measures sampling, not correctness.** It records that a situation occurred; whether the design behaved correctly in it is the business of the checkers. Coverage methodology openly *assumes* an orthogonal mechanism detects misbehaviour.

Break that assumption and coverage becomes vanity: a covergroup counting jobs whose outputs nobody compares is **a tally of unwitnessed events**.

> **AND WHY IT IS GAMEABLE**
>
> The goal is to remove bugs. Coverage closure is a well-defined, measurable **derivative** of that goal — and a derivative can be optimised directly. Stimulus tuned to tickle bins while checkers sleep converges on 100% *faster* than honest verification, having discarded the part that was slowing it down.

The cure is structural, not moral: review coverage models together with the checkers watching the same events, and make *"who fails if this scenario misbehaves?"* a mandatory review question for every coverpoint.

CHAPTER 6 · WORKED EXAMPLE

Bins at the behavioural cliffs — and two ways to fake it

Honest. The width bins follow the datapath, not the number line: behaviour bends at the 16-lane boundary and at the extremes.

```
cp_width : coverpoint job.tensor_w {
  bins single     = {1};        // degenerate geometry
  bins sub_lane   = {[2:15]};   // partial lane use
  bins lane_exact = {16};       // one full MAC pass
  bins lane_plus1 = {17};       // full pass + remainder
  bins multi_lane = {[18:255]}; // steady-state
  bins max_width  = {256};      // architectural max
  // Legal range is 1..256 in a 9-bit field: reaching the
  // datapath with anything else is a bug, not a hole.
  illegal_bins out_of_range = {0, [257:511]};
}
// precision x width-class x contention = 3 x 6 x 2 = 36
x_prec_width_dma : cross cp_prec, cp_width, cp_dma;
// channels interact with precision, not contention
x_prec_ch        : cross cp_prec, cp_ch;      // 3 x 5 = 15
```

Instead of one cross of 3 × 6 × 5 × 2 = **180**, two crosses totalling **51** — each defensible. The opening bug lives in `(PREC_2B, single, contended)` : a bin this model *demands*.

Vanity. Same attributes, same sampling call, same instant. It compiles, runs, and reaches 100% before the first coffee of the first regression.

```
cp_width : coverpoint job.tensor_w {
  bins lo_half = {[1:128]};
  bins hi_half = {[129:256]};
}
cp_dma : coverpoint dma_contended;
// no cross
```

- `lo_half` lumps the degenerate width 1, the lane boundary 16 and the remainder case 17 into one 128-value bin — filing the bug-exposing scenarios with their harmless neighbours
- **No cross:** "2-bit weights" and "contention" are each ticked without ever having occurred *together*

> **A THIRD WAY, NEEDING NO CHANGE TO THE BINS**
>
> Keep the honest model and read contention **live at job completion** instead of latching it across the streaming window. That measures whether the DMA happened to be busy at one irrelevant instant — and fills the bin either way.

Nothing in the report distinguishes any of these from the honest model. The metric is exactly as good as the model, and the model can only be judged by *reading it*.

Shaping: state the reachable space, with the reasons attached

The crossbar's route coverage — did every manager reach every subordinate? — is 5 × 4 = **20** combinations, trivially worth having. Then add what protocol verification cares about:

AXIS SET	CROSS SIZE
manager × subordinate	20
+ burst type × direction	**120**
+ 6 length buckets × 4 transfer sizes	2,880
+ 16 transaction IDs	46,080

WHY UNREACHABLE BINS ARE NOT HARMLESS PADDING

They make 100% unattainable — which forces late bulk exclusions and destroys the audit trail — or they normalise a dashboard that reads 61% forever, teaching everyone to stop looking.

```
x_route : cross cp_mgr, cp_sub, cp_burst, cp_dir {
  // Instruction port is read-only by construction.
  ignore_bins ifetch_wr = binsof(cp_mgr.core_i)
                       && binsof(cp_dir.wr);
  // Debug manager issues only INCR accesses.
  ignore_bins dbg_burst = binsof(cp_mgr.dbg)
                       && (binsof(cp_burst.fixed)
                        || binsof(cp_burst.wrap));
  // Writes to the boot ROM must be rejected: observing the
  // error response is a REQUIRED event, so it lives in its
  // own error-response covergroup, not as a hole here.
  ignore_bins rom_wr    = binsof(cp_sub.rom)
                       && binsof(cp_dir.wr);
}
```

Exclusions remove 12 + 16 + 15, overlapping in 5, so their union is **38**. What remains is **82 defensible combinations rather than 120** — in a model that never grew to 2,880.

SHAPING IS RELOCATION, NOT DELETION

The boot-ROM line split one lump into an *ignored* stimulus combination **and** a required error-response coverage point elsewhere. And the shaped cross is **falsifiable**: if the "read-only" instruction port ever produces a write, the protocol checkers scream.

SAY *The real tractability criterion is not the bin count. It is whether a reviewer can hold the model in their head.*

Closure: the loop, the hole analysis, and the endgame

Reading holes — four techniques

- **Aggregation** coalesces many point-holes into few larger regions
- **Partitioning** groups holes that share semantics
- **Projection** finds attribute values *never* observed in any combination — it outranks any single missing cross product, because it says a whole dimension of the stimulus is broken
- **Hamming distance** measures how close two holes are

Fifty scattered holes aggregate into *"no 2-bit-weight job has ever run under contention, at any geometry"*: one finding, one root cause, one action.

Classify, then tune or target

- **Valid hole** — an intended observation that has not happened: usually a stimulus weakness, possibly a design error suppressing the behaviour
- **Invalid hole** — a bug in the coverage model itself. Its fix is a *restriction on the model with a reason*, not a test
- **Tune** (bias the constraints) while holes come in families; **target** (write the test) when the survivors become singular

> **THE ENDGAME INVERTS THE ECONOMICS**
>
> As of **2012**, manually closing the last 20% of coverage costs up to **80%** of a verification team's time and resources. That cost is why closure automation is a product area, not a convenience — and every automation optimises the *derivative*, making the checking question more urgent, not less.

SAY *Stop when the reviewed model is full, the exclusion file is current, and the closure history is boring. Models legitimately evolve during closure — that is the process working, not failing.*

Honest exclusions

Every real project excludes coverage. Exclusion is not the failure mode. **Unreviewable** exclusion is.

#	RULE	WHAT IT LOOKS LIKE
1	Every exclusion carries its reason, **inline**	Not "waived" but *"instruction port is read-only by construction — see integration spec §3.2"*. The justification travels in the same artifact, so the review is a code review
2	Exclusions are **classified by kind**, because kinds differ in owner and expiry	*Structural* (tied parameters — expires if the parameterisation changes) · *specification* (expires if the spec revs) · *risk-accepted* (reachable but not worth the cost — expires never, so it needs the most senior signature)
3	The model distinguishes **"not my job" from "must never happen"**	Excluding a forbidden combination with `ignore_bins` rather than `illegal_bins` is a double loss: the space shrinks *and* the watchdog is dismissed
4	Exclusions are **versioned and re-derived**, not archaeological	The file lives with the RTL and the vplan, is re-audited when either changes, and appears at sign-off as a first-class document. Formal unreachability analysis can *prove* many structural exclusions

THE AUDITOR'S TEST

For any uncovered bin, can the team produce **within a minute** either a plan to cover it, or a written, attributed, still-valid reason not to? Anything else is 100% by redefinition — moving the goalposts and mowing the old penalty box.

SAY *Every real project ships with an exclusion file that was argued about, and the arguing is the value. A waiver nobody contested is a waiver nobody read.*

DISCUSSION

Questions for the room — the measurement

1. Open one of your covergroups. For each coverpoint, answer: **who fails if this sampled scenario misbehaves?** Any coverpoint without an answer is vanity.
2. Find a bin in your reports named `auto[...]`. What behavioural cliff is hiding inside it?
3. Take your exclusion file. Pick a line at random. Can anyone in the room **explain and defend it** — and is it still true?

FACILITATION *Question 1 is the single most productive review habit in this deck. It converts a coverage review into a checker review at no extra cost.*

07

Metrics-Driven Sign-off

Building the instruments, reading them honestly, and turning them into the one decision that cannot be un-made.

Five instruments — and none of them trustworthy alone

INSTRUMENT	WHAT IT TELLS YOU
1 · Per-feature status	The **master instrument**, denominated in the only currency that matters: the plan. Discharged / in progress / blocked / waived, per vplan row, with traceability from spec clause to result. Everything else is a *leading indicator* of this
2 · Coverage trends	Not the number — the *curve*. "The DMA is at 96%" is nearly information-free: alarming if it was 96% three weeks ago with stimulus running, encouraging if it was 78% last Monday
3 · Bug curves	Bug-open and bug-closure rates over time — the simplest trend report in the discipline, and the most misread instrument on the panel
4 · Regression health	Pass rate over time, plus the health of the machine: turnaround from check-in to result, farm utilisation, runtime per block. A runtime spike is actionable the week it appears and archaeology a month later
5 · Debug-time share	The quiet one, and the most often missing. Debug dominates verification time and **varies unpredictably between projects** — it is the one early warning the other four miss

THE TEST FOR EVERY PROPOSED METRIC

What decision changes if this number moves? If nothing changes, do not build it. Metrics must be architected and maintained like any other verification code.

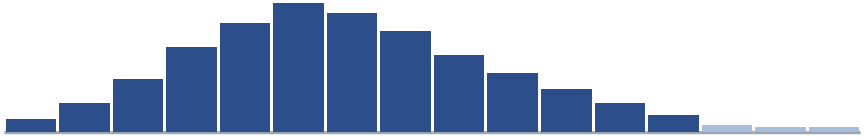
TWO RULES THAT KEEP THE PANEL HONEST

Count only clean runs — coverage from a failed simulation or a stale build is contamination, not evidence. **Version everything the numbers depend on** — RTL revision, testbench revision, tool versions, seed, configuration.

Two identical tails, two different truths

Bugs found per week. A discovery needs three things at once: **stimulus** that reaches the condition, a **checker** that catches it, an **engineer** who triages it into the tracker. The curve goes flat when bugs run out — or when any one of those three supply lines does.

Flat because clean

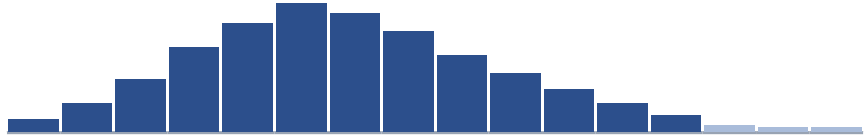


week 1 → 16 | shaded = the flat window

DURING THE FLAT WINDOW

New irritator sequences and new constraints went in. Coverage plateaued *at target*, holes dispositioned. Fresh-seed yield measured and ~zero. Checkers stable or strengthened. Triage queue empty. Last finds scattered, low severity, in modelled corners.

Flat because starved



week 1 → 16 | the same picture, exactly

DURING THE FLAT WINDOW

Same tests, more seeds — a fixed constraint set keeps re-sampling the same region, so the regression has **memorised its own answers**. Coverage plateaued *below* target. Checkers waived to "stabilise" the regression. A triage backlog hides undiagnosed discoveries.

SAY *Progress metrics are snapshots of what has happened — like a stock index or a batting average. They cannot be extrapolated, and the bug curve is where that limitation does the most damage.*

Six questions to ask a flat tail

QUESTION	FLAT BECAUSE CLEAN	FLAT BECAUSE BLIND
Was new stimulus added during the flat window — sequences, constraints, *irritators*?	Yes — and it found nothing	No — same tests, more seeds
Is functional coverage still moving?	Plateaued *at target*, holes dispositioned	Plateaued *below* target, or bins nobody reviewed
What is the fresh-seed yield (failures per thousand new seeds)?	Measured, and ~zero	Not measured, or seeds re-explore the same space
Did checker strength change?	Checkers stable or strengthened	Checkers waived or disabled to "stabilise" regression
Is the triage queue empty?	Yes — failures were dispositioned	A backlog hides undiagnosed discoveries
Where were the *last* bugs found?	Scattered, low severity, in modelled corners	Clustered, severe, in a corner the plan never modelled

THE LAST ROW DESERVES ITS OWN RULE

A severe bug found late, in a corner the coverage model did not describe, **is not one bug — it is a sample from a region your instruments were not watching.** The honest response is to treat it as evidence about the *model*: extend the coverage model and the stimulus around the find, and let the curve earn its flatness again.

SAY *One more curve belongs beside it: bug-open versus bug-closure. The gap is the debug backlog. The two lines must meet, not merely both go flat.*

The sign-off checklist

An **evidence aggregation**: every claim of doneness traced to an artifact a skeptic could re-derive. Written *before* the end — it is the plan's last column, not a document invented under deadline.

#	ITEM	WHAT SIGN-OFF ADDS
1	**Every plan row discharged or dispositioned**	With an executable plan, a database query rather than an interview. A row nobody can trace is undischarged, whatever anyone remembers
2	**Coverage closed or waived**	Every hole carries a waiver with five fields: **hole, reason, risk argument, owner, expiry**
3	**Static verification clean**	Lint, CDC and RDC with their waiver files *reviewed*, not just present
4	**Formal results, assumptions on the table**	A full proof under an over-constrained environment is a beautifully formatted blind spot. Read the assume list, not the checkmarks
5	**Open-bug disposition**	Fix / waive as errata (workaround named, doc owner assigned) / defer (with the argument). An open bug with no disposition is an unread instrument
6	**Regression health over a window**	A single green run proves the weather, not the climate — and only error-free runs on the tagged candidate contribute
7	**Reproducibility**	Every number regenerable from RTL and testbench revision, tool versions, configuration and seed

THREE OUTCOMES — AND NOTE THE MIDDLE ONE

Signed off · conditionally signed off · held. A conditional sign-off without a condition you can test is sign-off with a guilty conscience.

Three flat bug curves, three different verdicts

BLOCK	PLAN ROWS	FUNC COV	CODE COV	BUGS S1/S2/S3	7-DAY PASS	DISCOVERY (3WK)	FRESH STIMULUS (3WK)
the crossbar	41/41	100% →	98.7% →	0 / 0 / 1	100% →	flat	yes — new irritators, zero finds
the neural accelerator	37/38	98.9% ↑	97.8% ↑	0 / 0 / 2	99.8% ↑	flat	yes — quantization corners, zero finds
the DMA engine	44/47	96.0% →	98.2% →	0 / 1 / 3	99.2% →	flat	**no** — team in debug

SIGNED OFF

The crossbar. Corroborated on every axis: fresh stimulus found nothing, coverage at target, triage queue empty, last finds months old and scattered. Three formal properties are proven only to a bound — on the record. *Eleven minutes of the meeting.*

CONDITIONAL

The accelerator. One row undischarged: 2-bit weights × concurrent DMA traffic, which no block-level bench can generate. Waiver with all five fields — hole, reason, risk argument, owner (SoC verification lead), **expiry: closes in SoC regression by week 41, or the block returns to this room**.

HELD

The DMA. Three rows undischarged. Corroboration fails on four of six axes. And the disqualifier: the week-35 find — abort-and-resume corruption — had *no cross behind it*. Coverage there reads 71%, but that is not the hole: **the hole has no bins at all.**

SAY *The DMA is held two weeks with the work aimed at the blind spot, not the bug: abort-and-resume as a class, a cross of abort point × geometry × other-channel state, and a checker on the invariant the bug violated. The curve must earn its flatness under the new instruments.*

Escape analysis: five questions, answered in writing

Escapes are the statistical norm: **14%** first-silicon success, **87%** of FPGA projects with non-trivial production escapes (2024). An escape is a perfectly targeted audit of your process, bought at the worst price. Wasting it is the unforgivable part.

#	QUESTION	WORKED ON THE EVENT UNIT'S RETENTION ESCAPE
1	What was the bug? *Mechanism, not symptom*	A wake event arriving in the same cycle as isolation release corrupts the restore path of the retention flops
2	Why did our stimulus never reach it?	Power tests stepped through save/sleep/restore with *directed* timing; nothing randomized wake arrival against the power FSM
3	Why did no checker catch it — or could none have seen it?	Checks looked at configuration *after* restore completed, so the race window was unobserved — and plain RTL does not model state loss at all. **Checker gap** *and* **platform gap**
4	Why did our metrics say we were done?	The plan rows were discharged by those directed tests; the model had no cross of wake-timing × power-state transition. **The dashboard was faithful to a blind plan**
5	What changes? *For the class, never only the instance*	New plan-row family for randomized wake timing; the missing cross; power-aware simulation pulled earlier; and a new checklist item — *power-intent behaviour verified on a power-aware platform, not inferred from RTL*

THE NON-NEGOTIABLE GROUND RULE

The analysis is **blameless** — not out of kindness, but as instrumentation hygiene. The moment a post-mortem can end a career, every future post-mortem receives laundered data. Question 4's answer is often *"the metrics were fine and the plan was blind"* — obtainable only from an engineer who is not defending herself.

DISCUSSION

Questions for the room — sign-off

1. Your bug curve is flat this week. **Which of the six corroborating instruments can you actually produce today?** Whichever you cannot is the one to build first.
2. Take your last waiver. Does it have all five fields — hole, reason, risk argument, owner, **expiry**? An empty expiry makes it permanent by accident.
3. Name your organisation's last escape. Did it end as a **changed row in the plan and the checklist** — or as a slide nobody reopened?

FACILITATION *A mature team's checklist reads like a scar-tissue map: each odd-looking item is a former escape, filed where it can never surprise anyone again.*

PART II · CLOSING

What Part II established

- **A behaviour is verified only if it is in the plan.** Every downstream metric measures against the feature list, so extraction has the highest cost of omission in verification.
- **One row, one metric.** A row that needs two kinds of evidence is two rows — otherwise it closes by fatigue.
- **Coverage measures sampling, not correctness**, and "100%" is a statement about your model. Checkers are what keep the derivative attached to the goal.
- **Exclusions are legitimate only as reviewable artifacts**: classified, reasoned, attributed, versioned.
- **No single metric portrays the project.** Sign-off is the act of reading the instruments against each other — and then accepting the residue, on the record.

Read the chapters

5	Verification Planning — extraction, the vplan row, executable plans, the review, risk
6	Coverage: Theory and Practice — the taxonomy, model design, closure, honest exclusions
7	Metrics-Driven Sign-off — the instruments, bug curves, the checklist, escape analysis

> THE LOOP THAT CLOSES ON ITSELF
>
> The checklist is not a static document. It is the accumulated sediment of every post-mortem the organisation has survived — new rows in the two artifacts every future project must walk past: **the plan and the checklist**.

Next: **Part III — Dynamic Verification.** Testbench architecture, stimulus, UVM as a methodology, assertions, regression engineering.

SAY *Part I said completeness is impossible. Part II is what makes the alternative respectable: promises written down, evidence measured, residue accepted by name.*